K.R. MANGALAM UNIVERSITY

School of Engineering & Technology

PHASE-II CAPSTONE PROJECT REPORT

QUIZ MASTER

Interactive Timed Quiz Application in Kotlin

Course Code & Name	ENSP465 – New Age Programming Languages Lab
Semester & Academic Year	Semester VII, Academic Year 2023–2027
Faculty Name	Dr. Hemraj Kumawat
Affiliation	School of Engineering & Technology, K.R. Mangalam University
Submission Deadline	10 November 2026

TEAM MEMBERS

Name	Roll Number	Semester / Section
Vanshika Bhardwaj	2301730318	Semester VII, Section E
Taha Jamal	201730292	Semester VII, Section E
Dev Sood	2301730121	Semester VII, Section B
Akul Gulati	2301730136	Semester VII, Section B

GitHub Repository

https://github.com/vansh1ka23/NAPL-Vanshika-bhardwaj-2301730318

Acknowledgement

We would like to thank our faculty member, Dr. Hemraj Kumawat, and the School of Engineering & Technology at K.R. Mangalam University for the opportunity to work on this capstone project as part of the New Age Programming Languages Lab. Preparing this application helped us connect classroom learning with a practical software project. We also thank our teammates for contributing to the planning, development work and presentation of the project.

1. Project Abstract

Quiz Master is a simple Android quiz application built around short multiple-choice questions. The main idea is to give a student a quick way to attempt a quiz on a mobile phone and see the result without waiting for someone to check the answers manually. The app starts with a loading screen and a home screen. After starting the quiz, the user sees a question, a set of answer choices and a countdown timer. The interface then gives visual feedback for an answer and displays a score at the end.

The screenshots supplied with this report show the main stages of the application: loading, the start page, correct and incorrect answer feedback, and a final result page. One example result shown in the screenshots is 3 out of 5, along with the message “GOOD”. This report describes the visible behaviour of the application and the project information provided by the team.

2. Introduction

Quizzes are often used in classrooms to revise topics and check how well students have understood a subject. They are useful because a student can answer a few questions and quickly find out which areas they know well and which areas need more practice. A small mobile application can make this process easier to access and more convenient.

The Quiz Master project focuses on this basic idea. It presents questions in a clear format and gives the user a limited amount of time to answer. The interface is intentionally simple: the user does not have to move through complicated menus before starting a quiz. The project is developed using Kotlin, which is the programming language selected for the Kotlin-based application in the New Age Programming Languages Lab.

3. Problem Statement

A student who wants to practise a topic may not always have a printed question sheet, a teacher available to check answers, or a quick way to calculate a score. Even when questions are available, checking answers manually can take time and may interrupt the practice session.

The problem addressed by Quiz Master is therefore straightforward: provide a small, easy-to-use quiz application where a user can answer multiple-choice questions, work against a timer and see a result at the end. The project is not intended to replace a full learning management system. It is a focused application for short quiz attempts and immediate score feedback.

4. Aim and Objectives

The aim of the project is to build a timed multiple-choice quiz application for Android using Kotlin. The application should be simple enough for a first-time user to understand and should guide the user from the start screen to the final result.

- Create a mobile quiz application using Kotlin.
- Provide a clear start screen so the user can begin a quiz easily.
- Display a question with multiple answer options.
- Show a countdown timer while a question is being attempted.
- Make correct and incorrect answer feedback visually clear.
- Calculate and show the user's final score after the quiz.

• Keep the interface simple and readable on a mobile screen.

5. Scope of the Project

The current project scope is centred on the basic quiz journey shown in the supplied screenshots. It includes the app launch screen, a start screen, a timed question screen, feedback for correct and incorrect answers, and a result screen. These elements are enough to demonstrate the central purpose of the application.

Features such as user accounts, online leaderboards, cloud synchronisation, teacher dashboards and a large question-management system are not claimed as part of the current version in this report. They are possible additions for a later version if the team decides to extend the project.

6. Intended Users and Use Case

The main intended users are students who want to attempt short quizzes for practice. A user opens the application, selects the Start option and answers the questions displayed on the screen. The application provides feedback during the quiz and presents the score after the attempt is complete.

The application can be used for general knowledge or subject-based practice, depending on the questions included in the project. The screenshots contain sample questions about general knowledge, including the capital of India and the birthplace of cricket.

7. Main Features and Screen Description

7.1 Loading screen: The first screen shown in the supplied UI evidence is a loading screen. It displays the application's logo in the centre while the app opens. This gives the application a clear starting point and introduces its visual identity.

7.2 Home / start screen: The next screen displays the title “QUIZ MASTER”, the app logo and a large Start button. The layout keeps the main action easy to find. A user can understand what the app is for and begin without needing instructions.

7.3 Question screen and timer: The question screen displays a question near the centre and provides four answer options. A countdown timer is visible above the question. This makes the quiz time-based and encourages the user to answer within the time allowed.

7.4 Correct-answer feedback: One screenshot shows a correct option highlighted in green. This colour cue makes it easy to recognise that the selected answer was accepted as correct.

7.5 Incorrect-answer feedback: Another screenshot shows an incorrect selection highlighted in red. This provides a clear visual difference from the correct-answer state.

7.6 Result screen: At the end of the quiz, the result screen displays the score. The supplied example shows “3/5” and the message “GOOD”. This gives the user a quick summary of their performance.

8. Technologies and Tools

Kotlin: Kotlin is the selected programming language for the application. It is used for the app's programming logic and is the language relevant to the New Age Programming Languages Lab project topic.

Android application platform: The screenshots show the application running in an Android-style mobile interface. The project is intended as an Android app.

Development environment: Android Studio is the standard development environment for Kotlin Android applications. The exact project configuration, Android SDK version and dependency versions should be checked in the team's repository before they are included in a final technical record.

Repository: The source repository supplied by the team is linked in this report. It can be used to review the project files and source code.

9. How the Application Works

The application follows a short sequence of steps. First, the app opens and shows its loading screen. Once the home screen appears, the user selects Start. The quiz then presents a question with a set of answer choices and a visible countdown. The user selects an option. The interface indicates whether the answer is correct or incorrect. After the quiz questions have been attempted, the app displays the final score.

This flow keeps the experience predictable. Each screen has one main purpose, so the user can focus on answering the questions rather than figuring out how to use the application.

9.1 User Flow at a Glance

Stage	What the user sees or does
1. Open app	The loading screen appears.
2. Start quiz	The user sees the Quiz Master home screen and taps Start.
3. Answer question	A question, answer choices and countdown timer are displayed.
4. View feedback	The selected answer is shown with correct or incorrect visual feedback.
5. Finish quiz	The result screen displays the final score and a performance message.

10. Basic Logic Behind a Quiz

At a high level, a quiz application needs to keep track of the question being shown, the available answer choices, the correct answer and the user's score. When a user selects an option, the application compares that selection with the correct answer for the current question. If the selection matches, the score can be increased. The interface then displays the appropriate feedback.

A timer is also part of the visible question screen. Its purpose is to show how much time remains for the current question or quiz stage. The precise timer handling and the way the app moves between questions should be explained using the actual Kotlin source files during the project demonstration. This report does not assume implementation details that are not visible in the supplied material.

11. Functional Requirements

The application should open to its initial screen and provide a way to reach the quiz. It should display a question and its answer options, show a countdown, respond to a user's answer selection, distinguish correct and incorrect answers, and present a final score. These requirements describe the user-facing behaviour demonstrated by the screenshots.

- The user can start the quiz from the home screen.
- The app displays MCQ questions and answer choices.
- The app displays a timer on the question screen.
- The app gives a visible indication for correct and incorrect answers.
- The app shows a result at the end of the quiz.

12. Non-Functional Requirements

The application should be easy to understand, with readable text and clearly separated answer options. Buttons and answer choices should be large enough to select on a phone. Screen transitions should feel consistent, and the quiz should respond promptly when the user selects an answer. These are practical quality expectations for a small mobile quiz app.

13. Testing and Observations

The screenshots supplied by the team provide visual evidence of several application states. The table below records what can be observed in those screenshots. It is a visual review of the provided evidence, not a claim that every possible test case or device configuration has been independently tested.

Test / screen	What was checked in the supplied evidence	Observation
Loading screen	An initial screen is displayed when the app opens.	Visible
Home screen	The app title, logo and Start button are displayed.	Visible
Question screen	A question, four choices and countdown timer are displayed.	Visible
Correct answer	The correct option is visually highlighted in green.	Visible
Incorrect answer	The incorrect selected option is visually highlighted in red.	Visible
Result screen	A score and performance label are displayed.	Visible: 3/5, GOOD

14. Interface Screenshots

The following images were provided by the project team. They are included to show the appearance of the application and the main stages of a quiz attempt.

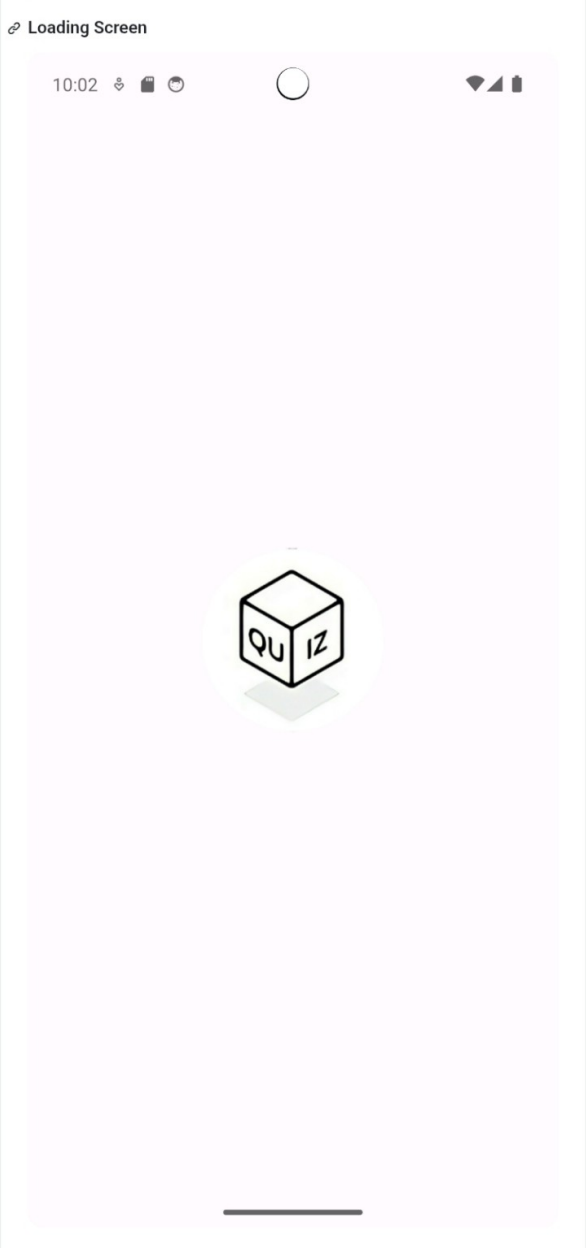


Figure 1: Loading screen



Figure 2: Quiz Master home screen

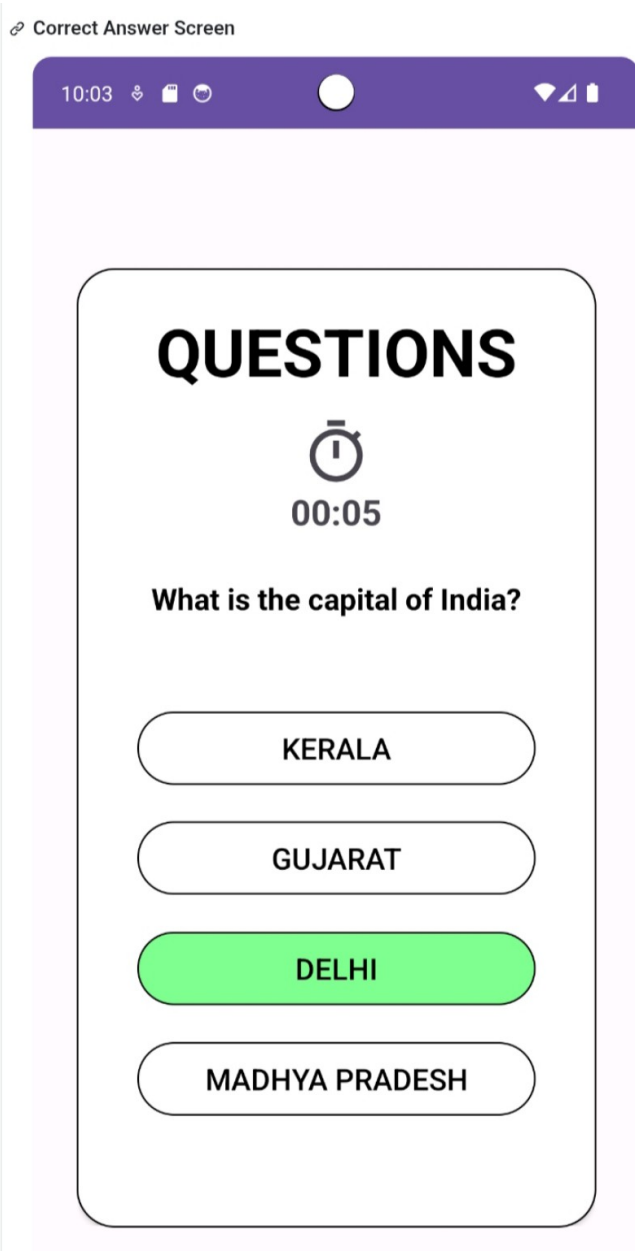


Figure 3: Correct-answer feedback



Figure 4: Incorrect-answer feedback



Figure 5: Final result screen

15. Current Limitations

The current report is based on the project title, repository link and screenshots shared by the team. The screenshots demonstrate the main quiz screens, but they do not show every internal detail of the application. For that reason, this report does not claim that the app includes features such as online accounts, a remote database, a leaderboard or an administrator panel.

The visible sample result is one example of a completed quiz. More testing would be needed to check how the application behaves with different question sets, when time runs out, when the user changes screens during a quiz, and on phones with different screen sizes.

16. Future Improvements

There are several useful ways the project could be improved in a later version. The team could add separate categories such as programming, science and general knowledge so that users can choose what they want to practise. It could also include different difficulty levels and allow the quiz duration to be adjusted.

Another useful improvement would be a review page that shows which questions were answered correctly and which were missed. Saving previous scores on the device would let a student compare attempts over time. If the project is extended further, a teacher or administrator could be given a way to add and update questions. These are suggestions for future work, not features claimed for the current version.

17. Conclusion

Quiz Master is a focused project that demonstrates how a simple quiz can be presented through an Android application using Kotlin. The screens supplied for the report show the user journey from opening the app to starting a quiz, answering questions, receiving feedback and viewing a final score.

The project is a useful starting point for practising application structure, screen design and basic quiz behaviour. It also leaves room for future additions, such as more question categories, score history and answer explanations. Overall, the application aims to make short quiz practice simple and accessible for students.

18. Repository and Project Reference

GitHub repository provided by the team: https://github.com/vansh1ka23/NAPL-Vanshika-bhardwaj-2301730318.

The repository address above has been copied from the information supplied by the team. The repository could not be opened during preparation of this expanded report, so the team should verify that the link is accessible and that the submitted repository contains the final version of the project. Please also check the source code before the viva so that the explanation of implementation details matches the actual code.

Prepared for Phase-II Capstone Project Submission | ENSP465